

Universidad de San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ciencias y Sistemas
Introducción a la Programación y Computación 2

Proyecto 1: Chapin Rescue

Ensayo de documentación técnica

Horacio José Lemus
Carne 202300345
Sección B

Introducción

El presente documento describe el análisis, diseño e implementación del sistema de control desarrollado para la empresa ficticia Chapin Warriors, S. A., en el marco del Proyecto 1 del curso de Introducción a la Programación y Computación 2. El sistema permite cargar la configuración de ciudades en conflicto a partir de archivos XML, y calcular estrategias de rescate de unidades civiles y extracción de recursos mediante robots autónomos, representando gráficamente los resultados con la herramienta Graphviz.

La solución fue construida en su totalidad utilizando el lenguaje C#, aplicando el paradigma de programación orientada a objetos y estructuras de datos propias desarrolladas específicamente para este proyecto, sin recurrir a las estructuras nativas del lenguaje como List, Queue o Stack, conforme lo exige el enunciado del proyecto.

Descripción general del problema

El problema plantea la simulación de un sistema de rescate y extracción en ciudades afectadas por conflictos bélicos. Un dron denominado ChapinEyes sobrevuela cada ciudad y construye un mapa bidimensional compuesto por celdas, donde cada celda puede representar un camino transitable, un punto de entrada, una unidad militar con una capacidad de combate determinada, una unidad civil a rescatar, un recurso a extraer, o una celda intransitable.

Sobre este mapa operan dos tipos de robots. Las unidades ChapinRescue ingresan por un punto de entrada y buscan un camino seguro hasta una unidad civil, evitando por completo cualquier celda ocupada por una unidad militar. Las unidades ChapinFighter, en cambio, poseen una capacidad de combate propia y pueden enfrentar unidades militares siempre que su capacidad sea mayor a la de estas, perdiendo en el proceso una cantidad de capacidad equivalente a la de la unidad vencida.

Análisis y diseño de la solución

Estructuras de datos propias

Dado que el enunciado prohíbe el uso de estructuras nativas de C# para el almacenamiento de datos, se diseñaron dos clases genéricas como base de todo el sistema: Nodo de T y ListaEnlazada

de T. La clase `Nodo` representa la unidad básica de una lista enlazada simple, almacenando un dato de tipo genérico y una referencia al siguiente nodo. La clase `ListaEnlazada` encapsula la lógica de inserción, recorrido y conteo de elementos, y es reutilizada a lo largo de todo el proyecto para almacenar colecciones de ciudades, robots, filas de celdas, y resultados intermedios de los algoritmos de búsqueda.

Esta decisión de diseño permitió construir una única estructura genérica reutilizable en lugar de crear una estructura distinta para cada tipo de colección, reduciendo la duplicación de código y centralizando el mantenimiento de la lógica de almacenamiento.

Modelo de datos: Celda, Ciudad y Robot

La clase `Celda` representa la unidad mínima de información del mapa, almacenando su posición (fila y columna), su tipo (camino, intransitable, punto de entrada, unidad civil, recurso o unidad militar), y su capacidad de combate cuando corresponde. Adicionalmente, la clase incorpora los atributos `Visitada`, `CeldaAnterior` y `EnCamino`, que son utilizados exclusivamente durante la ejecución de los algoritmos de búsqueda de caminos, permitiendo marcar el recorrido y reconstruir la ruta encontrada.

La clase `Ciudad` agrupa el nombre, las dimensiones y una malla de celdas organizada como una lista enlazada de listas enlazadas, es decir, una lista de filas donde cada fila es a su vez una lista de celdas. Esta representación evita el uso de arreglos bidimensionales nativos y respeta la restricción de utilizar únicamente estructuras propias del estudiante.

Para el modelado de los robots se aplicó herencia: la clase abstracta `Robot` concentra el atributo común `Nombre`, mientras que las clases `ChapinRescue` y `ChapinFighter` heredan de ella. Únicamente `ChapinFighter` incorpora el atributo adicional `Capacidad`, que representa su capacidad de combate y que disminuye durante la ejecución de una misión de extracción en la que se enfrenten unidades militares.

Carga de datos desde el archivo XML

La clase `LectorXML` es responsable de interpretar el archivo de configuración utilizando las clases del espacio de nombres `System.Xml` provisto por el lenguaje, exclusivamente para la lectura del archivo. Toda la información extraída es almacenada en las estructuras propias descritas

anteriormente. El proceso de carga se divide en tres etapas: la creación de cada ciudad a partir de su nombre y dimensiones, la construcción de la malla de celdas a partir del contenido de cada etiqueta fila, y la asignación de capacidad de combate a las celdas que corresponden a unidades militares, según las etiquetas `unidadMilitar` del archivo.

Durante el desarrollo se identificó que el texto de una fila podría llegar con una longitud menor a la esperada. Para dotar al sistema de mayor robustez, la construcción de cada fila se realiza iterando hasta el número de columnas declarado para la ciudad, en lugar de la longitud literal del texto leído, completando con espacios en blanco cuando sea necesario.

Algoritmo de búsqueda de caminos

El núcleo lógico del sistema es la clase `Misión`, que implementa el algoritmo de búsqueda en anchura, conocido como BFS por sus siglas en inglés, para resolver ambos tipos de misión. Se eligió BFS porque garantiza encontrar un camino válido explorando la malla nivel por nivel desde la celda de entrada, utilizando la propia `ListaEnlazada` como estructura de cola, insertando al final y procesando desde el inicio.

Para la misión de rescate, el algoritmo considera transitables únicamente las celdas de tipo entrada, camino y unidad civil, descartando cualquier celda con unidad militar. Para la misión de extracción, el algoritmo permite adicionalmente atravesar celdas con unidad militar siempre que la capacidad de combate del robot supere la capacidad de la unidad, sin restar la capacidad durante la exploración. Una vez identificado el camino definitivo hacia el recurso, se recorre la ruta completa desde el destino hasta el origen, y únicamente en ese momento se resta la capacidad real del robot por cada unidad militar efectivamente presente en el camino elegido. Esta decisión evita que el algoritmo penalice al robot por unidades militares exploradas en caminos alternativos que finalmente no fueron utilizados.

En ambos casos, cuando la cola de exploración se agota sin haber alcanzado el destino, el algoritmo retorna un valor nulo, que el programa interpreta como Misión Imposible.

Generación de gráficos con Graphviz

La clase `GeneradorGraphviz` recorre la malla completa de la ciudad y construye un archivo en formato DOT que representa cada celda como un nodo rectangular, coloreado según su tipo. Las

celdas que forman parte del camino encontrado por el algoritmo de búsqueda se resaltan en color dorado, replicando la convención visual establecida en el enunciado del proyecto. Para lograr que los nodos se organicen visualmente como una cuadrícula equivalente a la malla original, se emplean conexiones invisibles entre celdas consecutivas de una misma fila, y entre la primera celda de cada fila y la primera celda de la fila siguiente, aprovechando el atributo `rank=same` de Graphviz.

Interfaz de usuario

El sistema opera mediante un menú de consola secuencial que solicita al usuario la ruta del archivo de configuración, muestra las ciudades disponibles para su selección, y permite elegir entre una misión de rescate o de extracción. Cuando existe más de una unidad civil, recurso o robot compatible disponible, el sistema presenta las opciones y solicita la selección correspondiente; cuando solo existe una opción, esta se utiliza automáticamente sin solicitar confirmación adicional, conforme lo especifica el enunciado.

Pruebas realizadas

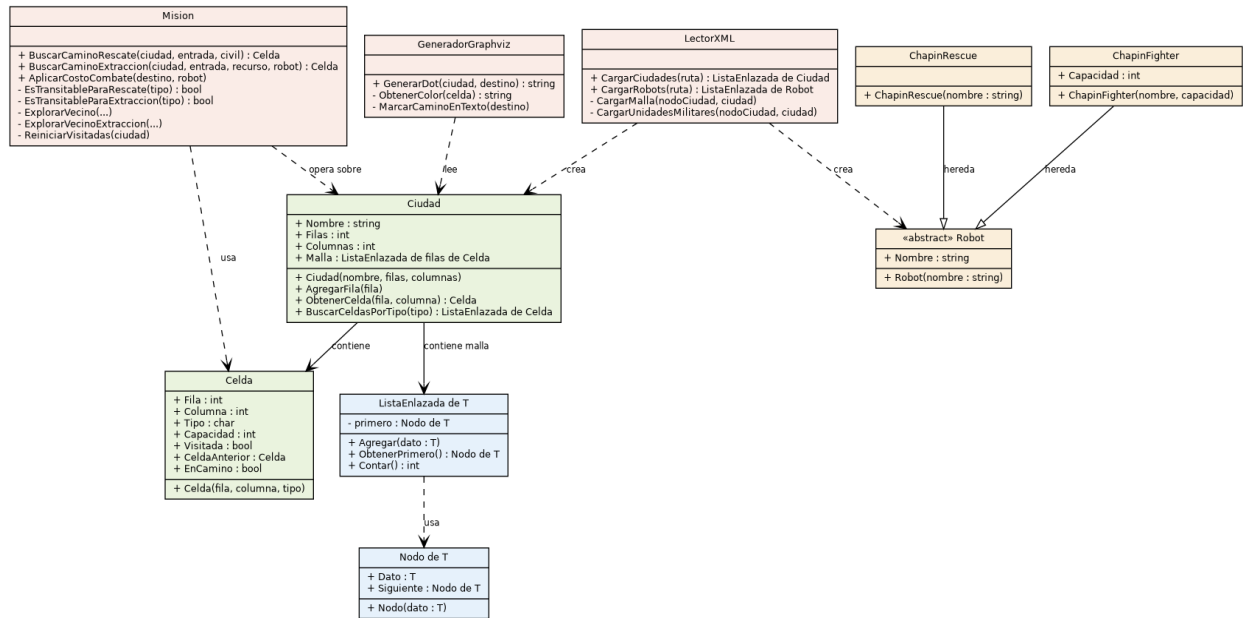
Se diseñaron y ejecutaron distintos archivos de configuración de prueba para validar el comportamiento del sistema ante casos especiales. Se confirmó el correcto funcionamiento del sistema ante: selección múltiple de unidades civiles y de robots del mismo tipo; ciudades sin unidades civiles o sin recursos disponibles, verificando que el sistema informe la situación sin generar errores; y la imposibilidad de completar una misión de extracción cuando la capacidad de combate del robot es insuficiente para vencer a la unidad militar que bloquea el único camino disponible. En todos los casos el sistema respondió de acuerdo con el comportamiento esperado según el enunciado del proyecto.

Conclusiones

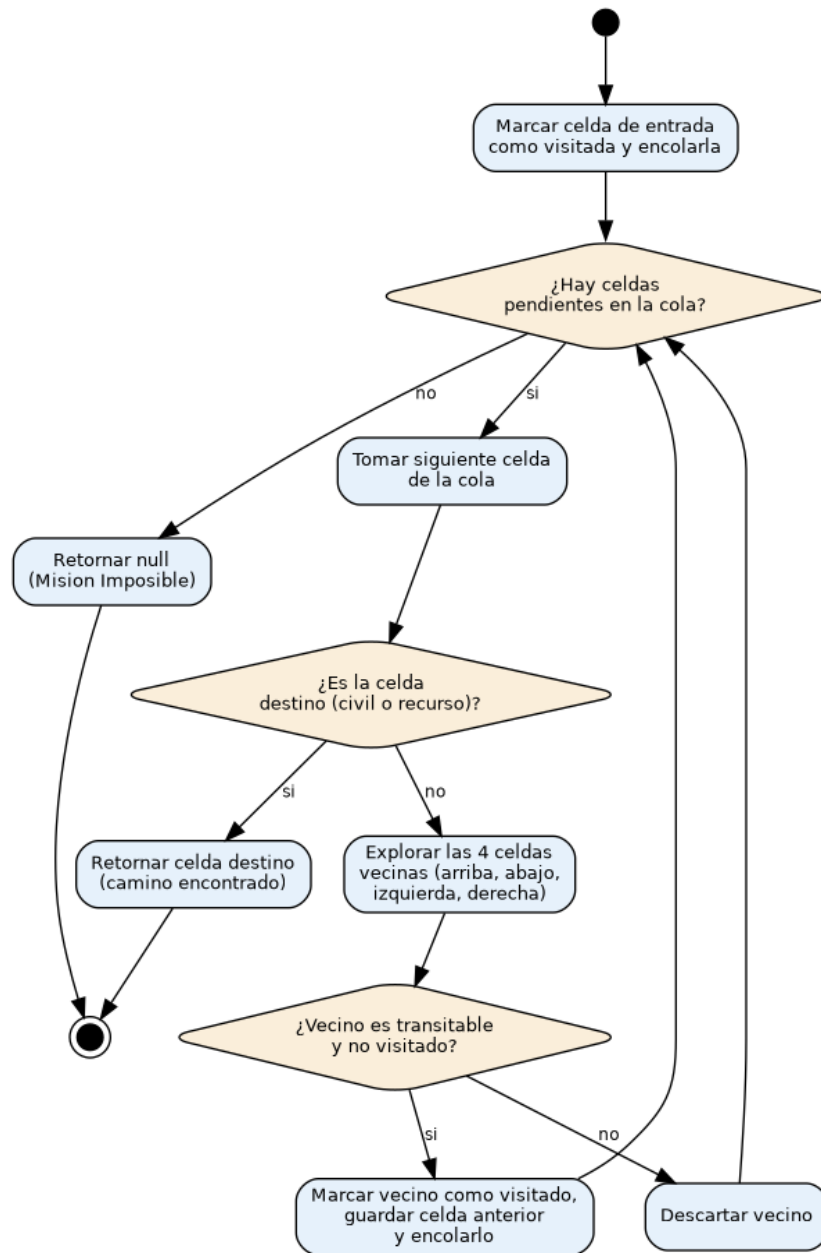
El desarrollo del proyecto permitió aplicar de forma integral los conceptos de programación orientada a objetos, estructuras de datos propias basadas en memoria dinámica, manipulación de

archivos XML y algoritmos de búsqueda sobre estructuras tipo grafo. La construcción de una estructura genérica reutilizable, como la clase ListaEnlazada de T, demostró ser una decisión de diseño que redujo significativamente la duplicación de código a lo largo del proyecto. Asimismo, la separación de responsabilidades entre las clases del modelo de datos, la lógica de búsqueda y la generación de gráficos facilitó el desarrollo incremental y las pruebas independientes de cada componente del sistema.

Anexo 1: Diagrama de clases



Anexo 2: Diagrama de actividad del algoritmo de búsqueda



Anexo 3: Diagrama de flujo del sistema

